

APIs & Web Services II

Authentication • Rate Limiting • OpenAPI / Swagger • API Integration

Session 6 of 7 | 4 Hours

Big Data Engineering Programme
Academic Year 2024–2025

Session Overview	
Duration	4 hours (theory + lab)
Theory	Auth (API keys, JWT), rate limiting, OpenAPI/Swagger, third-party integ
Lab	Secure the Session 5 API; add Swagger docs; consume a third-party API fo
Prerequisites	Session 5 API running
Tools used	Python 3, Flask, Flask-JWT-Extended, Flask-Limiter, flask-smorest

🎯 Learning Objectives

By the end of this session, students will be able to:

- ✔ Distinguish authentication from authorisation and implement API key and JWT Bearer token auth in Flask.
- ✔ Explain the structure of a JWT and verify its signature.
- ✔ Apply rate limiting to protect an API against abuse and configure per-endpoint limits.
- ✔ Write an OpenAPI 3.0 specification and serve an interactive Swagger UI from Flask.
- ✔ Consume a third-party REST API in Python, handle pagination and errors, and ingest the data into Kafka.
- ✔ Design a data enrichment pipeline that combines internal sensor data with external API data.

Contents

1	Part I – API Security: Authentication & Authorisation	3
1.1	Authentication vs Authorisation	3
1.2	API Key Authentication	3
1.2.1	How API Keys Are Transmitted	3
1.2.2	API Key Validation Flow	4
1.2.3	Pros and Cons of API Keys	4
1.3	JWT: JSON Web Tokens	4
1.3.1	JWT Structure	4
1.3.2	Standard JWT Claims	5
1.3.3	JWT Authentication Flow	5
1.4	Choosing an Authentication Strategy	6
2	Part II – Rate Limiting	7
2.1	What Is Rate Limiting?	7
2.2	Rate Limiting Algorithms	7
2.3	Rate Limit HTTP Headers	7
2.4	Rate Limiting in Flask with Flask-Limiter	8
3	Part III – OpenAPI and Swagger	9
3.1	What Is OpenAPI?	9
3.2	OpenAPI 3.0 Document Structure	9
3.3	Serving Swagger UI with flask-smorest	11
4	Part IV – Consuming Third-Party APIs	13
4.1	Why Consume External APIs?	13
4.2	The <code>requests</code> Library	13
4.3	Handling Pagination	13
4.4	Resilient HTTP Clients with Retry and Backoff	15
5	Part V – Lab Session	16
5.1	Step 0 – Install Dependencies	16
5.2	Step 1 – API Key Authentication	16
5.2.1	1a – Define the Key Store and Decorator	16
5.2.2	1b – Apply to Endpoints	18
5.3	Step 2 – JWT Authentication	19
5.3.1	2a – Configure Flask-JWT-Extended	19
5.3.2	2b – Login Endpoint (Issues JWT)	19
5.3.3	2c – JWT-Protected Endpoint	20
5.4	Step 3 – Rate Limiting	20
5.5	Step 4 – Swagger UI	21
5.6	Step 5 – Weather Enrichment Endpoint	22
5.6.1	5a – Weather Fetcher Helper	22
5.6.2	5b – Enrichment Endpoint	24
5.7	Step 6 – End-to-End Test	25

6 Summary & Key Takeaways	28
6.1 Vocabulary Checklist	28
6.2 Further Reading	28
6.3 Preview: Session 7	29
Appendix A – Complete Endpoint Reference	30
Appendix B – Glossary	31

1. Part I – API Security: Authentication & Authorisation

1.1 Authentication vs Authorisation

These two concepts are frequently confused but are fundamentally distinct:

lightbg Concept	Question answered	Example
Authentication (AuthN)	Who are you? Verify the identity of the caller.	API key, JWT token, username + password
Authorisation (AuthZ)	What are you allowed to do? Verify permissions <i>after</i> identity is confirmed.	Role “admin” can DELETE; role “reader” can GET only

Security Principle

Authentication always comes first: you must know *who* is making the request before you can decide *what* they are allowed to do. An unauthenticated request should return **401 Unauthorized**. An authenticated but unauthorised request returns **403 Forbidden**.

1.2 API Key Authentication

Definition – API Key

An **API key** is a long, randomly generated string that a client includes in every request to prove its identity. The server maintains a store (database or config file) mapping keys to client identities and permissions.

1.2.1 How API Keys Are Transmitted

lightbg Method	Example	Notes
Header (recommended)	X-API-Key: abc123	Invisible in logs, not cached by browsers
Query parameter	?api_key=abc123	Avoid: key appears in URL logs
Authorization header	Authorization: ApiKey abc123	Follows HTTP convention

Never transmit API keys (or any credentials) over plain HTTP. Always use HTTPS in production. A key transmitted over HTTP can be intercepted by any network observer between the client and server.

1.2.2 API Key Validation Flow

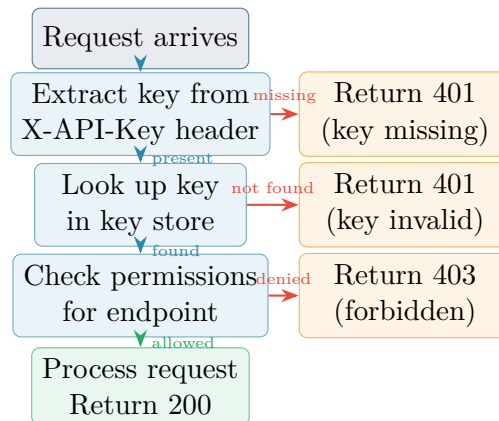


Figure 1: API key validation flow: extract, verify, authorise, serve.

1.2.3 Pros and Cons of API Keys

lightbg Advantages	Disadvantages
Simple to implement and use	No expiry by default
Works for server-to-server auth	Key is long-lived (rotation needed)
Stateless (no session store)	Shared secret (cannot identify user)
Easy to revoke per client	Not suitable for browser-based clients

1.3 JWT: JSON Web Tokens

Definition – JWT (JSON Web Token)

A **JWT** is a compact, URL-safe, *self-contained* token that encodes a set of claims (assertions about the user) and signs them cryptographically. The server can verify the token without any database lookup by checking the signature – making JWT ideal for stateless, distributed authentication.

1.3.1 JWT Structure

A JWT consists of three Base64URL-encoded parts separated by dots:

Header.Payload.Signature

1.3.2 Standard JWT Claims

lightbgb Claim	Name	Meaning
sub	Subject	Unique identifier of the token subject (user ID)
iss	Issuer	Who issued the token (e.g., <code>sensor-api</code>)
aud	Audience	Intended recipient(s) of the token
exp	Expiration	Unix timestamp after which the token is invalid
iat	Issued At	Unix timestamp when the token was created
nbf	Not Before	Token not valid before this timestamp
jti	JWT ID	Unique token identifier (for revocation)

⚠ Security Warning

JWT payload is NOT encrypted – only signed. Anyone can decode the Base64URL payload and read the claims. *Never* put sensitive data (passwords, credit card numbers, PII) in the JWT payload. Use JWE (JSON Web Encryption) if the payload must be confidential.

1.3.3 JWT Authentication Flow

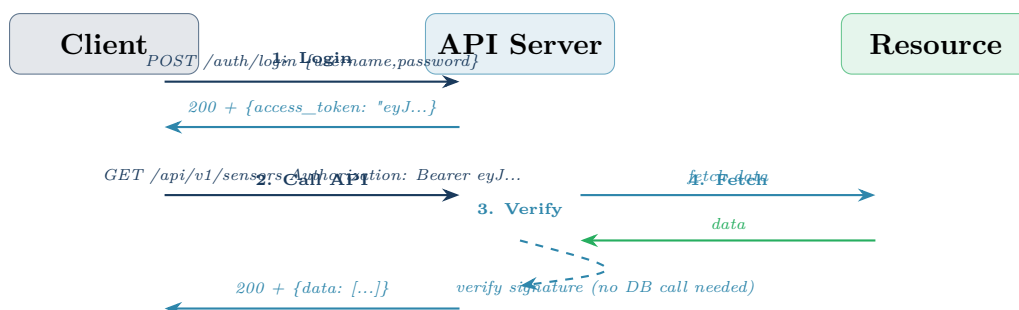


Figure 2: JWT flow: login once to get a token, then include token in every subsequent request. No server-side session needed.

1.4 Choosing an Authentication Strategy

Table 1: Authentication strategies compared

lightbg Strategy	Best for	Avoid when
API Key	Server-to-server; pipelines; long-running integrations	User-facing apps; short-lived sessions
JWT Bearer	User auth; microservices; short-lived tokens (15m–1h)	Very long sessions without re-fresh
OAuth 2.0	Delegated auth (“Login with Google”); third-party access	Simple internal APIs
Basic Auth	Simple scripts; internal tooling (over HTTPS only)	Public APIs; sensitive data

2. Part II – Rate Limiting

2.1 What Is Rate Limiting?

Definition – Rate Limiting

Rate limiting restricts the number of requests a client can make to an API within a given time window. It protects the API from accidental overload, deliberate abuse (DDoS), and runaway pipeline bugs that generate infinite requests.

2.2 Rate Limiting Algorithms

Table 2: Rate limiting algorithms compared

lightbg Algorithm	How it works	Trade-off
Fixed Window	Count requests in fixed time slots (e.g., 100/minute). Counter resets at window boundary.	Simple; allows burst at window edge
Sliding Window	Count requests in a rolling window. Smoother than fixed; more memory.	Fairer; more complex to implement
Token Bucket	Bucket fills at a fixed rate; each request consumes one token. Allows controlled bursting.	Most flexible; used by AWS API Gateway
Leaky Bucket	Requests queue up; processed at a fixed output rate. Queue overflow = reject.	Smooth output; latency for bursts

2.3 Rate Limit HTTP Headers

When rate limiting is active, APIs should communicate the current limits to clients via standard headers:

lightbg Header	Meaning
X-RateLimit-Limit	Maximum requests allowed in the window
X-RateLimit-Remaining	Requests remaining in the current window
X-RateLimit-Reset	Unix timestamp when the window resets
Retry-After	Seconds to wait before retrying (429 response)

i Rate Limit Response Code

When a client exceeds the limit, the server returns **429 Too Many Requests** with a **Retry-After** header. Clients must respect this header and implement *exponential backoff* – wait, double the wait, wait again – before retrying. Never poll after a 429; it only makes the problem worse.

2.4 Rate Limiting in Flask with Flask-Limiter

```
from flask import Flask
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

app = Flask(__name__)

# Identify clients by their IP address
limiter = Limiter(
    app=app,
    key_func=get_remote_address,
    default_limits=["200 per day", "50 per hour"],
    storage_uri="memory://", # use Redis in production
)

# Default limits apply to all routes
@app.route("/api/v1/sensors")
def list_sensors():
    ...

# Override limit for a specific route
@app.route("/api/v1/auth/login")
@limiter.limit("10 per minute")
def login():
    ...

# Exempt internal health check
@app.route("/api/v1/health")
@limiter.exempt
def health():
    ...
```

storage_uri="memory://": stores counters in process memory. This does not work across multiple server processes. For production with multiple workers, use **storage_uri="redis://localhost:6379"**.
key_func=get_remote_address: rate-limits per client IP. In a real API you would rate-limit per API key or per user ID for more accurate per-client enforcement.
@limiter.exempt: the health check endpoint must never be rate-limited, as load balancers call it every few seconds and would trigger 429 responses.

3. Part III – OpenAPI and Swagger

3.1 What Is OpenAPI?

Definition – OpenAPI Specification

The **OpenAPI Specification** (formerly Swagger) is a language-agnostic, machine-readable description of a REST API written in YAML or JSON. It defines endpoints, parameters, request bodies, response schemas, authentication methods, and examples. The current version is **OpenAPI 3.0**.

An OpenAPI document enables:

- **Interactive documentation** (Swagger UI / ReDoc)
- **Client SDK generation** in any language
- **Server stub generation**
- **Contract testing** – verify the API matches its spec
- **API gateway integration** (AWS API Gateway, Kong)

3.2 OpenAPI 3.0 Document Structure

```
openapi: "3.0.3"
info:
  title: Sensor Data API
  version: "1.0"
  description: REST API exposing IoT sensor readings and
    statistics.

servers:
  - url: http://localhost:5000/api/v1
    description: Local development server

paths:
  /sensors/{sensor_type}/latest:
    get:
      summary: Get latest reading for a sensor type
      tags: [Sensors]
      parameters:
        - name: sensor_type
          in: path
          required: true
          schema:
            type: string
            enum: [temperature, humidity, pressure]
        - name: n
          in: query
          required: false
```

```
    schema:
      type: integer
      minimum: 1
      maximum: 50
      default: 1
  responses:
    "200":
      description: Latest reading retrieved successfully
      content:
        application/json:
          schema:
            $ref: "#/components/schemas/SuccessResponse"
    "404":
      description: Sensor type not found
      content:
        application/json:
          schema:
            $ref: "#/components/schemas/ErrorResponse"

components:
  schemas:
    SensorReading:
      type: object
      required: [sensor, value, timestamp]
      properties:
        sensor:
          type: string
          example: temperature
        value:
          type: number
          example: 28.5
        unit:
          type: string
          example: "C"
        timestamp:
          type: integer
          description: Epoch milliseconds
          example: 1705312800000
        is_anomaly:
          type: boolean
          example: false

    SuccessResponse:
      type: object
      properties:
        status:
          type: string
          example: success
        count:
          type: integer
        data:
```

```

        oneOf:
          - $ref: "#/components/schemas/SensorReading"
          - type: array
            items:
              $ref: "#/components/schemas/SensorReading"

ErrorResponse:
  type: object
  properties:
    status:
      type: string
      example: error
    code:
      type: integer
      example: 404
    message:
      type: string

securitySchemes:
  ApiKeyAuth:
    type: apiKey
    in: header
    name: X-API-Key
  BearerAuth:
    type: http
    scheme: bearer
    bearerFormat: JWT

security:
  - ApiKeyAuth: []

```

\$ref: "#/components/schemas/...": reusable schema objects defined once and referenced everywhere. This prevents schema drift – if the SensorReading schema changes, all endpoints that reference it are updated automatically.

components/securitySchemes: declares the available auth methods. The top-level security block applies them globally. Individual endpoints can override or remove security requirements.

3.3 Serving Swagger UI with flask-smorest

```

from flask import Flask
from flask_smorest import Api

app = Flask(__name__)

# Configure OpenAPI
app.config["API_TITLE"] = "Sensor Data API"
app.config["API_VERSION"] = "v1"
app.config["OPENAPI_VERSION"] = "3.0.3"
app.config["OPENAPI_URL_PREFIX"] = "/"
# Swagger UI served at /swagger-ui

```

```
app.config["OPENAPI_SWAGGER_UI_PATH"] = "/swagger-ui"  
app.config["OPENAPI_SWAGGER_UI_URL"] = \  
    "https://cdn.jsdelivr.net/npm/swagger-ui-dist/"  
  
api = Api(app)
```

After this configuration, <http://localhost:5000/swagger-ui> serves an interactive Swagger UI where users can browse endpoints, see request/response schemas, and **execute live requests directly from the browser** – an invaluable tool for API consumers and QA teams.

4. Part IV – Consuming Third-Party APIs

4.1 Why Consume External APIs?

Data pipelines rarely operate in isolation. Common patterns for consuming third-party APIs include:

- **Data enrichment:** augment internal data with external context (weather data enriching sensor readings, exchange rates enriching financial data).
- **Reference data ingestion:** load lookup tables (country codes, product catalogues, user profiles) from upstream services.
- **Event triggers:** poll an external API for events and publish them to Kafka.
- **Data validation:** validate internal data against an authoritative external source.

4.2 The requests Library

Python's `requests` library is the de facto standard HTTP client for consuming REST APIs:

```
import requests

# Simple GET
response = requests.get(
    "https://api.open-meteo.com/v1/forecast",
    params={
        "latitude": 48.8566,      # Paris
        "longitude": 2.3522,
        "current": "temperature_2m,wind_speed_10m",
        "hourly": "temperature_2m",
    },
    timeout=10,                  # seconds; ALWAYS set this
)

# Always check status code before using data
response.raise_for_status()      # raises HTTPError on 4xx/5xx
data = response.json()          # parse JSON body
print(data["current"]["temperature_2m"])
```

timeout=10: always set a timeout. Without it, a slow external API will block your thread indefinitely, eventually hanging the entire application.

response.raise_for_status(): raises `requests.HTTPError` if the status code is 4xx or 5xx. This is cleaner than manually checking `response.status_code`.

response.json(): parses the response body as JSON. Raises `requests.JSONDecodeError` if the body is not valid JSON – always wrap in a try/except.

4.3 Handling Pagination

Most APIs that return collections implement **pagination** to avoid returning millions of records in one response. Common strategies:

lightbg Strategy	Parameters	Notes
Page-based	page=2&per_page=50	Simple; page numbers can shift if data changes
Offset-based	offset=100&limit=50	Equivalent to page-based; same limitations
Cursor-based	cursor=abc123&limit=50	Stable; ideal for streaming; used by Twitter/X
Link header	Link: <...?page=3>; rel="next"	Follow rel=next until absent

```
def fetch_all_pages(base_url: str, params: dict) -> list:
    """
    Generic paginated fetch using page-based pagination.
    Stops when the response returns an empty 'data' list
    or when no more pages are indicated.
    """
    all_records = []
    page = 1

    while True:
        params["page"] = page
        params["per_page"] = 100

        response = requests.get(base_url, params=params,
                                timeout=10)
        response.raise_for_status()
        body = response.json()

        records = body.get("data", [])
        if not records:
            break  # no more data

        all_records.extend(records)
        page += 1

        # Respect rate limits: check headers
        remaining = int(response.headers.get(
            "X-RateLimit-Remaining", 1))
        if remaining == 0:
            import time
            reset_at = int(response.headers.get(
                "X-RateLimit-Reset", 0))
            wait = max(0, reset_at - time.time()) + 1
            time.sleep(wait)

    return all_records
```

4.4 Resilient HTTP Clients with Retry and Backoff

Network errors are inevitable. Use urllib3 retry logic via `requests.adapters`:

```
import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

def create_resilient_session(
    total_retries: int = 3,
    backoff_factor: float = 0.5,
) -> requests.Session:
    """
    Returns a requests.Session that automatically retries
    on connection errors and 429/500/503 responses.

    backoff_factor: wait = backoff_factor * (2 ** retry_number)
    retry 0: 0s, retry 1: 1s, retry 2: 2s, retry 3: 4s
    """
    session = requests.Session()

    retry = Retry(
        total=total_retries,
        backoff_factor=backoff_factor,
        status_forcelist=[429, 500, 502, 503, 504],
        allowed_methods=["GET", "POST"],
        raise_on_status=False,
    )

    adapter = HTTPAdapter(max_retries=retry)
    session.mount("https://", adapter)
    session.mount("http://", adapter)

    return session
```

status_forcelist=[429, 500, ...]: automatically retry on these status codes. 429 = rate limited (back off and retry); 500/503 = transient server error.

backoff_factor=0.5: retries after 0s, 1s, 2s. With **backoff_factor=1:** 0s, 2s, 4s, 8s. Never use 0 – it hammers the server during an outage.

allowed_methods=["GET", "POST"]: by default, only GET is retried because POST is not idempotent. Add POST explicitly only if your POST endpoint is idempotent (e.g., it uses a client-provided ID).

5. Part V – Lab Session

Lab 6 – Secure the API, Add Swagger, Consume External Data

Duration 90 minutes

Objective Extend the Session 5 API with: API key auth, JWT login, rate limiting, Swagger UI,

Tools Flask, Flask-JWT-Extended, Flask-Limiter, flask-smorest, requests

Difficulty  Intermediate

Lab Timeline

Time	Activity	Goal
0–10 min	Install new dependencies	Env ready
10–25 min	Step 1: API key authentication	Protect all endpoints
25–40 min	Step 2: JWT login + Bearer token	User-level auth
40–50 min	Step 3: Rate limiting	429 in action
50–65 min	Step 4: Swagger UI	Live docs
65–80 min	Step 5: Weather enrichment endpoint	External API consumption
80–90 min	Step 6: End-to-end test	All features working

5.1 Step 0 – Install Dependencies

```
source venv/bin/activate
pip install flask-jwt-extended flask-limiter \
    flask-smorest marshmallow requests
```

flask-jwt-extended: adds JWT creation, decoding, and `@jwt_required()` decorator to Flask.

flask-limiter: rate limiting with pluggable storage backends.

flask-smorest: OpenAPI 3.0 spec generation + Swagger UI.

marshmallow: schema validation and serialisation (used by flask-smorest for request/response models).

requests: the standard Python HTTP client for consuming external APIs.

5.2 Step 1 – API Key Authentication

5.2.1 1a – Define the Key Store and Decorator

Add to `sensor_api/auth.py`:

```
"""
sensor_api/auth.py
Authentication: API key guard and JWT configuration.
"""
from functools import wraps
from flask import request, jsonify, g
import secrets
```

```
#           API Key store (use a database in production)

# Map: api_key -> {client_name, role}
API_KEYS = {
    "dev-key-sensor-read-only":
        {"client": "dashboard", "role": "reader"},
    "dev-key-sensor-write":
        {"client": "pipeline", "role": "writer"},
    "dev-key-admin":
        {"client": "admin", "role": "admin"},
}

def generate_api_key() -> str:
    """Generate a cryptographically secure 32-byte API key."""
    return secrets.token_urlsafe(32)

def require_api_key(f):
    """
    Decorator: validates X-API-Key header.
    Stores client info in Flask g on success.
    Returns 401 if key is missing or invalid.
    """
    @wraps(f)
    def decorated(*args, **kwargs):
        key = request.headers.get("X-API-Key")

        if not key:
            return jsonify({
                "status": "error",
                "code": 401,
                "message": "X-API-Key header is required.",
            }), 401

        client = API_KEYS.get(key)
        if not client:
            return jsonify({
                "status": "error",
                "code": 401,
                "message": "Invalid API key.",
            }), 401

        # Store client info in Flask g (required for
        # flask-smorest compatibility)
        g.api_client = client
        return f(*args, **kwargs)

    return decorated
```

```
def require_role(role: str):
    """
    Decorator factory: checks that g.api_client has the given
    role.
    Must be used AFTER @require_api_key.
    """
    def decorator(f):
        @wraps(f)
        def decorated(*args, **kwargs):
            client = g.get("api_client", {})
            allowed = {
                "reader": ["reader", "writer", "admin"],
                "writer": ["writer", "admin"],
                "admin": ["admin"],
            }
            if client.get("role") not in allowed.get(role, []):
                return jsonify({
                    "status": "error",
                    "code": 403,
                    "message": f"Role '{role}' or higher
                        required.",
                }), 403
            return f(*args, **kwargs)
        return decorated
    return decorator
```

secrets.token_urlsafe(32): generates a 32-byte cryptographically secure random string – much better than `uuid4()` or `random.token_hex()` for security tokens.

@wraps(f): preserves the original function's `__name__` and docstring. Without this, Flask's routing system would see multiple routes mapped to a function named “decorated”.

Role hierarchy: “reader” can call reader-or-above endpoints.

Why Flask g? When using flask-smorest's `@blp.route()` decorator, the Blueprint inspects function signatures to generate OpenAPI schemas. Passing extra keyword arguments (like `api_client`) causes a `TypeError` because the Blueprint tries to map them to path/query parameters. Flask's request context `g` is the correct way to share data between decorators and view functions in this scenario. “writer” can call writer-or-above. “admin” can call everything. This avoids duplicating permission checks in each route.

5.2.2 1b – Apply to Endpoints

In `app.py`, add the decorator to protected routes:

```
from auth import require_api_key, require_role

@blp.route(f"{API_PREFIX}/sensors")
@require_api_key
def list_sensors():
    # g.api_client = {"client": "dashboard", "role": "reader"}
    ...

@blp.route(f"{API_PREFIX}/readings", methods=["POST"])
```

```
@require_api_key
@require_role("writer")
def create_reading():
    # Only writer and admin keys can POST
    ...
```

5.3 Step 2 – JWT Authentication

5.3.1 2a – Configure Flask-JWT-Extended

```
from flask_jwt_extended import (
    JWTManager, create_access_token,
    jwt_required, get_jwt_identity, get_jwt,
)
from datetime import timedelta

app.config["JWT_SECRET_KEY"] = "CHANGE-ME-IN-PRODUCTION"
app.config["JWT_ACCESS_TOKEN_EXPIRES"] = timedelta(hours=1)

jwt = JWTManager(app)
```

JWT_SECRET_KEY: the HMAC secret used to sign tokens. In production, this must be a long random string stored in an environment variable or secrets manager – never hard-coded.

JWT_ACCESS_TOKEN_EXPIRES=timedelta(hours=1): access tokens expire after 1 hour. Clients must re-authenticate and get a new token. Short expiry reduces the window of exposure if a token is compromised.

5.3.2 2b – Login Endpoint (Issues JWT)

```
# Simulated user store (use a database in production)
USERS = {
    "alice": {"password": "secret123", "role": "admin"},
    "bob": {"password": "pass456", "role": "reader"},
}

@app.route(f"{API_PREFIX}/auth/login", methods=["POST"])
@limiter.limit("10 per minute")
def login():
    """
    POST /api/v1/auth/login
    Body: {"username": "alice", "password": "secret123"}
    Returns: {"access_token": "eyJ..."}
    """
    body = request.get_json(silent=True)
    if not body:
        return jsonify({"status": "error",
                        "message": "JSON body required."}), 400

    username = body.get("username", "")
```

```

password = body.get("password", "")
user      = USERS.get(username)

# Constant-time comparison prevents timing attacks
import hmac
if not user or not hmac.compare_digest(
    user["password"], password):
    return jsonify({"status": "error",
                    "message": "Invalid credentials."}), 401

# Create JWT with custom claims
token = create_access_token(
    identity=username,
    additional_claims={"role": user["role"]},
)
return jsonify({
    "status": "success",
    "access_token": token,
    "token_type": "Bearer",
    "expires_in": 3600,
}), 200

```

hmac.compare_digest(a, b): performs a *constant-time* string comparison. A naive `==` comparison exits early on the first differing character, leaking timing information that can be used to guess passwords character by character.

additional_claims={"role": ...}: embeds the user role directly in the JWT payload. The protected endpoint can read the role without a database lookup using `get_jwt()["role"]`.

5.3.3 2c – JWT-Protected Endpoint

```

@app.route(f"{API_PREFIX}/me")
@jwt_required()
def me():
    """
    GET /api/v1/me
    Returns identity and role from the JWT.
    Requires: Authorization: Bearer <token>
    """
    identity = get_jwt_identity()      # value of 'sub' claim
    claims   = get_jwt()               # full payload dict
    return jsonify({
        "status": "success",
        "data": {
            "username": identity,
            "role":     claims.get("role"),
        },
    }), 200

```

5.4 Step 3 – Rate Limiting

```

from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

limiter = Limiter(
    app=app,
    key_func=get_remote_address,
    default_limits=["500 per day", "100 per hour"],
    storage_uri="memory://",
    headers_enabled=True,    # adds X-RateLimit-* headers
)

# Strict limit on login (brute-force protection)
@app.route(f"{API_PREFIX}/auth/login", methods=["POST"])
@limiter.limit("10 per minute; 3 per second")
def login():
    ...

# Relaxed limit on read-only endpoints
@app.route(f"{API_PREFIX}/sensors")
@limiter.limit("300 per minute")
def list_sensors():
    ...

# No limit on health check
@app.route(f"{API_PREFIX}/health")
@limiter.exempt
def health():
    ...

```

headers_enabled=True: adds X-RateLimit-Limit, X-RateLimit-Remaining, and X-RateLimit-Reset headers to every response. API consumers can use these to implement proactive rate limit handling without waiting for a 429.

"10 per minute; 3 per second": Flask-Limiter allows multiple limits on one endpoint using semicolons. Both must be satisfied. This prevents bursts of 10 requests in one second even though the per-minute limit would allow it.

5.5 Step 4 – Swagger UI

```

# Add to app.py
app.config.update({
    "API_TITLE": "Sensor Data API",
    "API_VERSION": "v1",
    "OPENAPI_VERSION": "3.0.3",
    "OPENAPI_URL_PREFIX": "/",
    "OPENAPI_SWAGGER_UI_PATH": "/swagger-ui",
    "OPENAPI_SWAGGER_UI_URL": "https://cdn.jsdelivr.net/npm/swagger-ui-dist/",
    "OPENAPI_REDOC_PATH": "/redoc",
})

```

```
from flask_smorest import Api
api = Api(app)
```

After adding this configuration and restarting the server, navigate to:

- <http://localhost:5000/swagger-ui> – Interactive Swagger UI
- <http://localhost:5000/redoc> – ReDoc (read-only, cleaner for public docs)
- <http://localhost:5000/openapi.json> – Raw OpenAPI JSON spec

In Swagger UI you can authenticate with your API key (click the **Authorize** button), then execute live requests directly from the browser without writing any code.

5.6 Step 5 – Weather Enrichment Endpoint

5.6.1 5a – Weather Fetcher Helper

Create `sensor_api/weather_utils.py`:

```
"""
sensor_api/weather_utils.py
Fetches current weather from the Open-Meteo free API.
No API key required for Open-Meteo.
Docs: https://open-meteo.com/en/docs
"""

import logging
import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

log = logging.getLogger(__name__)

OPEN_METEO_URL = "https://api.open-meteo.com/v1/forecast"

# Known city coordinates
CITIES = {
    "paris": (48.8566, 2.3522),
    "london": (51.5074, -0.1278),
    "berlin": (52.5200, 13.4050),
    "new_york": (40.7128, -74.0060),
}

_session = None

def _get_session() -> requests.Session:
    """Resilient session with retry and backoff."""
    global _session
    if _session is None:
        retry = Retry(
            total=3, backoff_factor=0.5,
            status_forcelist=[429, 500, 503],
        )
        _session = requests.Session()
        _session.mount("https://", HTTPAdapter(max_retries=retry))
    return _session
```

```

        _session = requests.Session()
        _session.mount("https://",
                        HTTPAdapter(max_retries=retry))
    return _session

def get_weather(city: str = "paris") -> dict:
    """
    Fetch current weather for a city from Open-Meteo.

    Args:
        city: one of 'paris', 'london', 'berlin', 'new_york'

    Returns:
        dict with temperature_2m, wind_speed_10m,
              weather_code, city, latitude, longitude

    Raises:
        ValueError: if city is unknown
        requests.HTTPError: if API call fails
    """
    city = city.lower().replace(" ", "_")
    if city not in CITIES:
        raise ValueError(
            f"Unknown city '{city}'. "
            f"Valid: {sorted(CITIES.keys())}"
        )

    lat, lon = CITIES[city]
    session = _get_session()

    log.info("Fetching weather for %s (%.4f, %.4f)",
            city, lat, lon)

    resp = session.get(
        OPEN_METEO_URL,
        params={
            "latitude": lat,
            "longitude": lon,
            "current": (
                "temperature_2m,relative_humidity_2m,"
                "wind_speed_10m,weather_code"
            ),
        },
        timeout=8,
    )
    resp.raise_for_status()
    data = resp.json()
    current = data.get("current", {})

    return {

```

```

    "city":            city,
    "latitude":        lat,
    "longitude":       lon,
    "temperature_c":   current.get("temperature_2m"),
    "humidity_pct":    current.get("relative_humidity_2m"),
    "wind_speed_kmh":  current.get("wind_speed_10m"),
    "weather_code":    current.get("weather_code"),
    "fetched_at":      current.get("time"),
}

```

Open-Meteo is a free, open-source weather API that requires no API key or registration – ideal for teaching. It provides current, hourly, and daily forecasts for any latitude/longitude.

Resilient session with Retry: the same pattern as in the theory section. Every external API call must be wrapped in retry logic; production pipelines that consume APIs without retries will fail intermittently.

timeout=8: 8 seconds. Open-Meteo is fast, but any external service can become slow. Without a timeout, a slow API blocks the Flask worker thread, exhausting the thread pool and making the entire API unresponsive.

5.6.2 5b – Enrichment Endpoint

Add to app.py:

```

from weather_utils import get_weather, CITIES

@app.route(f"{API_PREFIX}/enriched/<sensor_type>")
@require_api_key
def enriched_reading(sensor_type: str):
    """
    GET /api/v1/enriched/{sensor_type}?city=paris
    Returns the latest sensor reading enriched with
    current weather data from Open-Meteo.

    Query parameters:
        city (str): paris | london | berlin | new_york
                    (default: paris)
    """
    err = validate_sensor(sensor_type)
    if err:
        return err

    city = request.args.get("city", "paris").lower()
    if city not in CITIES:
        return error(
            f"Unknown city '{city}'. "
            f"Valid: {sorted(CITIES.keys())}",
            400,
        )

    #         Fetch sensor reading

    try:

```

```

        readings = get_latest_readings(sensor_type, n=1)
    except Exception as exc:
        app.logger.error("Kafka error: %s", exc)
        return error("Failed to fetch sensor reading.", 503)

    sensor_data = readings[0] if readings else None

    #         Fetch weather

    try:
        weather = get_weather(city)
    except Exception as exc:
        app.logger.error("Weather API error: %s", exc)
        # Partial degradation: return sensor data without weather
        return success({
            "sensor": sensor_data,
            "weather": None,
            "warning": "Weather service unavailable.",
        })

    #         Return enriched payload

    return success({
        "sensor": sensor_data,
        "weather": weather,
        "context": {
            "outdoor_vs_sensor_temp": (
                round(
                    (sensor_data or {}).get("value", 0)
                    - (weather.get("temperature_c") or 0),
                    2,
                )
                if sensor_type == "temperature" else None
            ),
        },
    })

```

Partial degradation: if the weather API fails, we still return the sensor data with a `warning` field instead of failing the whole request with 503. This is the *graceful degradation* pattern – external dependencies should never bring down the core functionality of your service.

outdoor_vs_sensor_temp: a simple enrichment insight – the difference between the indoor sensor temperature and the outdoor weather temperature. This is the *business value* of combining data sources.

5.7 Step 6 – End-to-End Test

```

# Start the API
python sensor_api/app.py &

```

```
#          1. Test without auth (expect 401)

curl -s http://localhost:5000/api/v1/sensors | python3 -m
    json.tool

#          2. Test with valid API key

curl -s http://localhost:5000/api/v1/sensors \
    -H "X-API-Key: dev-key-sensor-read-only" \
    | python3 -m json.tool

#          3. POST with reader key (expect 403)

curl -s -X POST http://localhost:5000/api/v1/readings \
    -H "X-API-Key: dev-key-sensor-read-only" \
    -H "Content-Type: application/json" \
    -d '{"sensor":"temperature","value":29.5}' \
    | python3 -m json.tool

#          4. POST with writer key (expect 201)

curl -s -X POST http://localhost:5000/api/v1/readings \
    -H "X-API-Key: dev-key-sensor-write" \
    -H "Content-Type: application/json" \
    -d '{"sensor":"temperature","value":29.5}' \
    | python3 -m json.tool

#          5. JWT login

TOKEN=$(curl -s -X POST \
    http://localhost:5000/api/v1/auth/login \
    -H "Content-Type: application/json" \
    -d '{"username":"alice","password":"secret123"}' \
    | python3 -c "import sys,json; \
        print(json.load(sys.stdin)['access_token'])"
echo "Token: ${TOKEN:0:20}..."

#          6. Use JWT to call /me

curl -s http://localhost:5000/api/v1/me \
    -H "Authorization: Bearer $TOKEN" \
    | python3 -m json.tool

#          7. Test rate limiting

for i in {1..15}; do
    curl -s http://localhost:5000/api/v1/auth/login \
        -X POST -H "Content-Type: application/json" \
        -d '{"username":"x","password":"x"}' \
        | python3 -c "import sys,json; \
            d=json.load(sys.stdin); \
```

```
        print(f'[{i}] status={d["status"]}')
done

#           8. Enriched endpoint

curl -s \
  "http://localhost:5000/api/v1/enriched/temperature?city=paris" \
  -H "X-API-Key: dev-key-sensor-read-only" \
  | python3 -m json.tool

#           9. Swagger UI

echo "Open: http://localhost:5000/swagger-ui"
```

Reflection Questions

Answer before Session 7:

1. A JWT with `exp=now+1h` is stolen by an attacker. What options does the API server have to invalidate it before it expires? What are the trade-offs of each?
2. Your API is rate-limited to 100 requests/minute per IP. A legitimate pipeline runs 5 Spark executors on the same host, each calling the API concurrently. What problem does this create and how would you solve it?
3. The Open-Meteo API is unavailable. Explain two different strategies for handling this gracefully in the `/enriched` endpoint, with different trade-offs.
4. In the OpenAPI spec, why is it important to define `$ref: "#/components/schemas/..."` for the `SensorReading` schema rather than duplicating the schema in every endpoint that returns it?
5. A new requirement asks the API to support both API key auth and JWT auth on the same endpoint. How would you modify the `require_api_key` decorator to also accept a valid JWT Bearer token?

6. Summary & Key Takeaways

📌 What We Covered Today

1. **AuthN vs AuthZ**: authentication confirms identity (401 if failed); authorisation confirms permission (403 if denied). Always in that order.
2. **API keys** are simple, long-lived, shared secrets – ideal for server-to-server pipelines.
3. **JWT** tokens are self-contained, short-lived, and verifiable without a database lookup – ideal for user-facing apps and microservices.
4. **Rate limiting** protects the API from abuse and overload. Return 429 with a **Retry-After** header; clients must implement exponential backoff.
5. **OpenAPI 3.0** provides machine-readable API documentation, enables SDK generation, and serves interactive Swagger UI. Always document your APIs.
6. **Consuming external APIs** requires: timeouts on every call, automatic retry with backoff, pagination handling, and graceful degradation when the external service is unavailable.
7. In the lab, we secured the Session 5 API with API keys and JWT, added rate limiting, served Swagger UI, and built a weather enrichment endpoint that combines Kafka data with an external API.

6.1 Vocabulary Checklist

- | | |
|--------------------------------------|----------------------------------|
| ✓ Authentication (AuthN) | ✓ Token bucket / sliding window |
| ✓ Authorisation (AuthZ) | ✓ OpenAPI 3.0 / Swagger |
| ✓ API key | ✓ \$ref / components / schemas |
| ✓ JWT (Header / Payload / Signature) | ✓ Swagger UI / ReDoc |
| ✓ JWT claims (sub, exp, iat, role) | ✓ requests.Session / HTTPAdapter |
| ✓ Bearer token | ✓ Retry with backoff |
| ✓ OAuth 2.0 | ✓ Pagination (page / cursor) |
| ✓ Rate limiting | ✓ Graceful degradation |
| ✓ 429 Too Many Requests | ✓ Partial response |
| ✓ Retry-After / exponential backoff | ✓ Data enrichment |

6.2 Further Reading

- JWT specification: RFC 7519 – tools.ietf.org/html/rfc7519

- Flask-JWT-Extended docs: flask-jwt-extended.readthedocs.io
- OpenAPI 3.0 spec: spec.openapis.org/oas/v3.0.3
- OWASP API Security Top 10: owasp.org/www-project-api-security
- *requests* library docs: requests.readthedocs.io

6.3 Preview: Session 7

Final session – End-to-End Streaming Architecture:

- Combine all components: Kafka, REST API, Spark Streaming, and the data lake into one cohesive architecture.
- Orchestrate the full pipeline: ingest → transform → serve → monitor.
- Group project presentations: code walkthrough and performance analysis.

Appendix A – Endpoint Reference (Sessions 5 + 6)

lightbg Verb	URL	Description	Auth
GET	/api/v1/health	Health check (no auth)	None
POST	/api/v1/auth/login	Get JWT access token	None
GET	/api/v1/me	JWT identity + role	JWT
GET	/api/v1/sensors	List sensor types	API Key
GET	/api/v1/sensors/{type}/latest	Latest Kafka reading	API Key
GET	/api/v1/sensors/{type}/stats	Daily Parquet stats	API Key
POST	/api/v1/readings	Publish to Kafka	API Key (writer)
GET	/api/v1/enriched/{type}	Sensor + weather	API Key
GET	/swagger-ui	Interactive API docs	None
GET	/openapi.json	Raw OpenAPI spec	None

Appendix B – Glossary

lightbg Term	Definition
Authentication	Verifying who the caller is (identity).
Authorisation	Deciding what an authenticated caller is allowed to do.
API Key	Long random string used to authenticate a client.
Bearer Token	Token type; included as <code>Authorization: Bearer <token></code> .
Claims	Assertions inside a JWT payload (sub, exp, role, etc.).
Exponential backoff	Retry strategy: wait time doubles on each failure.
Graceful degradation	Returning partial results when a dependency fails.
HMAC	Hash-based Message Authentication Code used for JWT HS256.
JWT	JSON Web Token: signed, self-contained auth token.
OAuth 2.0	Delegated authorisation framework (“Login with Google”).
OpenAPI	Machine-readable REST API specification format.
Rate limiting	Capping requests per client per time window.
Retry-After	HTTP header indicating when to retry after a 429.
Swagger UI	Interactive HTML documentation served from an OpenAPI spec.
Token bucket	Rate limiting algorithm that allows controlled bursting.
429	HTTP status: Too Many Requests (rate limit exceeded).